

PAW

Binary Packet Specification

18-Byte Little-Endian LoRa Payload Format

Reference Encoding & Decoding Implementation

Version 1.0

September 2026

1. Overview

This document specifies the 18-byte binary payload transmitted by the PAW collar over LoRa P2P, and provides a reference implementation for both sides of the link: the C encoder used in the collar firmware, and a Python decoder suitable for a gateway or backend receiving the packets. All multi-byte fields use little-endian byte order.

2. Packet Layout

Byte	Field	Type	Description
0	Device ID	uint8	Unique collar identifier
1	Protocol Version	uint8	Packet format version
2	Status Flags	uint8	Bit 0: GPS valid Bit 1: motion-triggered wake
3–6	Latitude	int32	Degrees × 1,000,000
7–10	Longitude	int32	Degrees × 1,000,000
11–14	Timestamp	uint32	Seconds since boot
15–16	Battery	uint16	Battery voltage, millivolts
17	Sequence	uint8	Packet sequence number (wraps at 256)

Total payload size: 18 bytes. Encoding: little-endian.

3. Worked Example

The sample field values below encode to the following 18-byte payload, shown as hex octets:

Field	Sample Value
device_id	1
protocol_version	1
flags	3 (GPS valid = 1, motion wake = 1)
latitude	22.864000
longitude	88.401000
timestamp	123456
battery_mv	3785
sequence	42

Encoded bytes (hex):

```
01 01 03 80 E0 5C 01 68 E4 44 05 40 E2 01 00 C9 0E 2A
```

18 bytes, little-endian. Byte 0 = 01 (device_id) ... byte 17 = 2A (sequence = 42).

4. Reference Encoder (C — Collar Firmware)

This is the exact packing routine used on the collar (RUI3 / Arduino-compatible C++), reproduced from collar_firmware.cpp:

```
// Encodes the current GPS / battery / sequence state into an 18-byte
// little-endian PAW packet. Matches collar_firmware.cpp exactly.

static inline void writeLE32(uint8_t *buf, int32_t value) {
    buf[0] = value & 0xFF;
    buf[1] = (value >> 8) & 0xFF;
    buf[2] = (value >> 16) & 0xFF;
    buf[3] = (value >> 24) & 0xFF;
}

static inline void writeLE16(uint8_t *buf, uint16_t value) {
    buf[0] = value & 0xFF;
    buf[1] = (value >> 8) & 0xFF;
}

static uint8_t buildPacket(uint8_t *packet, bool motionWake) {
    int32_t lat_e6 = (int32_t)(g_gps_lat * 1000000.0);
    int32_t lon_e6 = (int32_t)(g_gps_lon * 1000000.0);

    packet[0] = DEVICE_ID;
    packet[1] = PROTOCOL_VERSION;

    packet[2] = 0;
    if (g_gps_valid) packet[2] |= (1 << 0);
    if (motionWake) packet[2] |= (1 << 1);

    writeLE32(&packet[3], lat_e6);
    writeLE32(&packet[7], lon_e6);
    writeLE32(&packet[11], (int32_t)g_gps_timestamp);
    writeLE16(&packet[15], g_battery_mv);
    packet[17] = g_sequence++; // wraps at 256 by design (uint8_t)

    return 18; // PACKET_LEN
}
```

Encoder: field values → 18-byte little-endian buffer.

5. Reference Decoder (C — Receiver / Gateway)

The receiving side inverts buildPacket() field-for-field: readLE32/readLE16 reassemble the little-endian byte groups, and the two status-flag bits are tested individually. PawPacket holds the result in the same field order as the packet layout in Section 2.

```
// Decodes an 18-byte PAW packet back into field values.
// Mirrors buildPacket() byte-for-byte, in reverse.
```

```

static inline int32_t readLE32(const uint8_t *buf) {
    return (int32_t)((uint32_t)buf[0] |
                    ((uint32_t)buf[1] << 8) |
                    ((uint32_t)buf[2] << 16) |
                    ((uint32_t)buf[3] << 24));
}

static inline uint16_t readLE16(const uint8_t *buf) {
    return (uint16_t)(buf[0] | (buf[1] << 8));
}

typedef struct {
    uint8_t device_id;
    uint8_t protocol_version;
    bool gps_valid;
    bool motion_wake;
    int32_t lat_e6; // degrees x 1,000,000
    int32_t lon_e6; // degrees x 1,000,000
    uint32_t timestamp; // seconds since boot
    uint16_t battery_mv;
    uint8_t sequence;
} PawPacket;

static void decodePacket(const uint8_t *packet, PawPacket *out) {
    out->device_id = packet[0];
    out->protocol_version = packet[1];

    out->gps_valid = (packet[2] & (1 << 0)) != 0;
    out->motion_wake = (packet[2] & (1 << 1)) != 0;

    out->lat_e6 = readLE32(&packet[3]);
    out->lon_e6 = readLE32(&packet[7]);
    out->timestamp = (uint32_t)readLE32(&packet[11]);
    out->battery_mv = readLE16(&packet[15]);
    out->sequence = packet[17];
}

```

Decoder: 18-byte little-endian buffer → PawPacket struct. Exact inverse of buildPacket().

6. Line-by-Line Walkthrough

This section traces every line of buildPacket() and decodePacket() against the exact worked-example values from Section 3, so each transformation from field value ↔ byte is shown explicitly rather than asserted.

6.1 Encoding: buildPacket()

Code	What happens with this packet's data
lat_e6 = (int32_t)(g_gps_lat * 1000000.0);	22.864000 × 1,000,000 = 22864000 (0x015CE080). Stored LE → bytes[3..6] = 80 E0 5C 01.
lon_e6 = (int32_t)(g_gps_lon * 1000000.0);	88.401000 × 1,000,000 = 88401000 (0x0544E468). Stored LE → bytes[7..10] = 68 E4 44 05.
packet[0] = DEVICE_ID;	device_id = 1 → byte[0] = 01.
packet[1] = PROTOCOL_VERSION;	protocol_version = 1 → byte[1] = 01.

Code	What happens with this packet's data
<code>packet[2] = 0; if (gps_valid) packet[2] = (1<<0); if (motionWake) packet[2] = (1<<1);</code>	Both true → 0b00000001 0b00000010 = 0b00000011 → byte[2] = 03.
<code>writeLE32(&packet[3], lat_e6);</code>	22864000 written least-significant-byte-first: 80, E0, 5C, 01 → bytes[3..6].
<code>writeLE32(&packet[7], lon_e6);</code>	88401000 written LSB-first: 68, E4, 44, 05 → bytes[7..10].
<code>writeLE32(&packet[11], (int32_t)g_gps_timestamp);</code>	123456 = 0x0001E240 → LSB-first: 40, E2, 01, 00 → bytes[11..14].
<code>writeLE16(&packet[15], g_battery_mv);</code>	3785 = 0x0EC9 → LSB-first: C9, 0E → bytes[15..16].
<code>packet[17] = g_sequence++;</code>	sequence = 42 = 0x2A → byte[17] = 2A. g_sequence becomes 43 for the next packet.
<code>return 18;</code>	Full frame: 01 01 03 80 E0 5C 01 68 E4 44 05 40 E2 01 00 C9 0E 2A — matches Section 3 exactly.

6.2 Decoding: decodePacket()

Code	What happens with this packet's data
<code>out->device_id = packet[0];</code>	byte[0] = 01 → device_id = 1.
<code>out->protocol_version = packet[1];</code>	byte[1] = 01 → protocol_version = 1.
<code>out->gps_valid = (packet[2] & (1<<0)) != 0;</code>	byte[2] = 03 = 0b011. Bit 0 set → gps_valid = true.
<code>out->motion_wake = (packet[2] & (1<<1)) != 0;</code>	Bit 1 of 0b011 also set → motion_wake = true.
<code>out->lat_e6 = readLE32(&packet[3]);</code>	Bytes 80 E0 5C 01 reassembled LSB-first → 0x015CE080 = 22864000 → lat = 22.864000°.
<code>out->lon_e6 = readLE32(&packet[7]);</code>	Bytes 68 E4 44 05 → 0x0544E468 = 88401000 → lon = 88.401000°.
<code>out->timestamp = (uint32_t)readLE32(&packet[11]);</code>	Bytes 40 E2 01 00 → 0x0001E240 = 123456 seconds.
<code>out->battery_mv = readLE16(&packet[15]);</code>	Bytes C9 0E → 0x0EC9 = 3785 mV.
<code>out->sequence = packet[17];</code>	byte[17] = 2A = 42, matching the value the encoder wrote.

Result: decodePacket() reproduces device_id, protocol_version, both flag bits, latitude, longitude, timestamp, battery_mv, and sequence exactly as they were before encoding — the two functions are byte-for-byte inverses of each other.